## Cloning

↳ 14 cells hidden

## Training Self Supervised model

↳ 1 cell hidden

## Using Self Supervised Weights

↳ 13 cells hidden

## Using Pretrained Weights

```
#remove after

gated_pt_path ='/content/gated_lorentz_part_v1.pt'
```

```
# Model configurations
pretrained_model_config = LorentzParTConfig(
    num_classes=10,
    embed_dim=128,
    num_heads=8,
    num_layers=8,
    num_cls_layers=2,
    num_mlp_layers=0,
    hidden_dim=256,
    hidden_mv_channels=8,
    in_s_channels=None,
    out_s_channels=None,
    hidden_s_channels=16,
    attention={},
    mlp={},
    dropout=0.1,
    expansion_factor=4,
    max_num_particles=128,
    num_particle_features=4,
    pair_embed_dims=[64, 64, 64],
    weights=gated_pt_path # replaced here
)
```

## Training Pretrained Model on saved gated weight

```
# Initialize the classifier model
pretrained_model = LorentzParT(config=pretrained_model_config).to(device)
pretrained_model
```

```
LorentzParT(
  (processor): ParticleProcessor()
  (encoder): LorentzParTEncoder(
    (equilinear): EquiLinear()
    (proj): Linear(in_features=16, out_features=128, bias=True)
    (interaction_embed): InteractionEmbedding(
      (embed): Sequential(
        (0): BatchNorm1d(4, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (1): Conv1d(4, 64, kernel_size=(1,), stride=(1,))
        (2): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (3): GELU(approximate='none')
        (4): Conv1d(64, 64, kernel_size=(1,), stride=(1,))
        (5): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (6): GELU(approximate='none')
        (7): Conv1d(64, 64, kernel_size=(1,), stride=(1,))
        (8): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (9): GELU(approximate='none')
        (10): Conv1d(64, 8, kernel_size=(1,), stride=(1,))
        (11): BatchNorm1d(8, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (12): GELU(approximate='none')
      )
    )
    (encoder): ModuleList(
```

```
      (0-7): 8 x ParticleAttentionBlock(
        (layernorm1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
        (pmha): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=128, out_features=128, bias=True)
        )
        (layernorm2): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
        (feedforward): Feedforward(
          (layernorm1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
          (linear1): Linear(in_features=128, out_features=512, bias=True)
          (act): GELU(approximate='none')
          (dropout1): Dropout(p=0.1, inplace=False)
          (layernorm2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
          (linear2): Linear(in_features=512, out_features=128, bias=True)
          (dropout2): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (fc): Linear(in_features=16384, out_features=16, bias=True)
  (equilinear): EquiLinear()
  (decoder): ModuleList(
    (0-1): 2 x ClassAttentionBlock(
      (layernorm1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
      (mha): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=128, out_features=128, bias=True)
      )
      (layernorm2): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
      (dropout): Dropout(p=0.0, inplace=False)
      (feedforward): Feedforward(
        (layernorm1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
        (linear1): Linear(in_features=128, out_features=512, bias=True)
        (act): GELU(approximate='none')
        (dropout1): Dropout(p=0.0, inplace=False)
        (layernorm2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
```

```
# Count parameters in the model
num_params = sum(p.numel() for p in pretrained_model.parameters() if p.requires_grad)
num_params
```

```
2271120
```

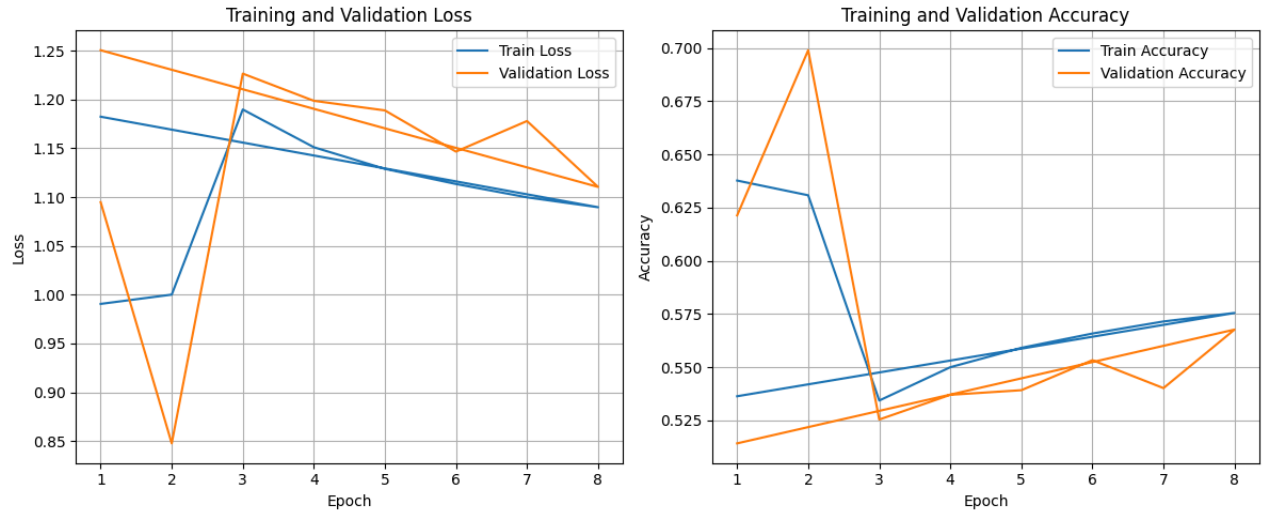
```
# Training configurations
pretrained_config = TrainConfig(
    batch_size=64,
    criterion={
        'name': 'cross_entropy_loss',
        'kwargs': {
            'reduction': 'mean'
        }
    },
    optimizer={
        'name': 'adam',
        'kwargs': {
            'lr': 1e-4
        }
    },
    scheduler={
        'name': 'exponential_lr',
        'kwargs': {
            'gamma': 0.95
        }
    },
    callbacks=[{
        'name': 'early_stopping',
        'kwargs': {
            'monitor': 'val_loss',
            'mode': 'min',
            'patience': 5
        }
    }],
    num_epochs=20,
    start_epoch=0,
    logging_dir=str(LOG_DIR),
    logging_steps=1000,
    save_best=True,
    save_ckpt=True,
    save_fig=False,
    device='cuda',
    num_workers=0,
    pin_memory=True
)
```

```
# Initialize the trainer
trainer = Trainer(
    model=pretrained_model,
    train_dataset=train_dataset,
    val_dataset=val_dataset,
    test_dataset=test_dataset,
    device=device,
    metric=accuracy_metric_ce,
    config=pretrained_config
)
```

```
#run2
# Train the model
pretrained_history, pretrained_model = trainer.train()
```

```
Training:  40%                                    100000/250000 [1:59:17<2:04:20, 20.11it/s, epoch=8/20, avg_loss=1.0897, avg_metric=0.5755]
step: 1000/250000 | train_loss: 1.3188 | train_metric: 0.4989
step: 2000/250000 | train_loss: 1.2609 | train_metric: 0.5211
step: 3000/250000 | train_loss: 1.2271 | train_metric: 0.5358
step: 4000/250000 | train_loss: 1.1917 | train_metric: 0.5513
step: 5000/250000 | train_loss: 1.1568 | train_metric: 0.5666
step: 6000/250000 | train_loss: 1.1251 | train_metric: 0.5802
step: 7000/250000 | train_loss: 1.0969 | train_metric: 0.5923
step: 8000/250000 | train_loss: 1.0735 | train_metric: 0.6026
step: 9000/250000 | train_loss: 1.0518 | train_metric: 0.6118
step: 10000/250000 | train_loss: 1.0317 | train_metric: 0.6203
step: 11000/250000 | train_loss: 1.0144 | train_metric: 0.6277
step: 12000/250000 | train_loss: 0.9982 | train_metric: 0.6345
epoch: 1/20 | val_loss: 1.0948 | val_metric: 0.6213
step: 13000/250000 | train_loss: 1.5339 | train_metric: 0.3967
step: 14000/250000 | train_loss: 1.3895 | train_metric: 0.4552
step: 15000/250000 | train_loss: 1.3448 | train_metric: 0.4746
step: 16000/250000 | train_loss: 1.3032 | train_metric: 0.4945
step: 17000/250000 | train_loss: 1.2469 | train_metric: 0.5214
step: 18000/250000 | train_loss: 1.1946 | train_metric: 0.5453
```

```
# Visualize the training history of the pretrained model
plot_history(pretrained_history)
```




```
step: 44000/250000 | train_loss: 1.1545 | train_metric: 0.5489
step: 45000/250000 | train_loss: 1.1538 | train_metric: 0.5489
step: 46000/250000 | train_loss: 1.1534 | train_metric: 0.5490
step: 47000/250000 | train_loss: 1.1532 | train_metric: 0.5490
step: 48000/250000 | train_loss: 1.1531 | train_metric: 0.5491
step: 49000/250000 | train_loss: 1.1513 | train_metric: 0.5498
```

```
#run2
# Train the model
pretrained_history, pretrained_model = trainer.train()
```

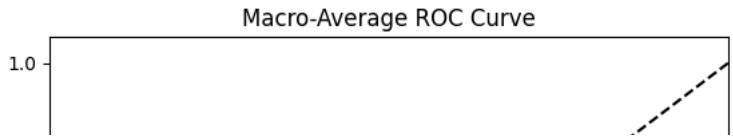
```
step: 54000/250000 | train_loss: 1.1370 | train_metric: 0.5556
Training:  5%                                     12500/250000 [19:07<3:17:01, 20.09it/s, epoch=1/20, avg_loss=1.1825, avg_metric=0.5364]
step: 55000/2                                                  tric: 0.5558
step: 56000/250000 | train_loss: 1.1342 | train_metric: 0.5569
step: 57000/250000 | train_loss: 1.1336 | train_metric: 0.5574
step: 58000/250000 | train_loss: 1.1332 | train_metric: 0.5578
step: 59000/250000 | train_loss: 1.1312 | train_metric: 0.5581
step: 60000/250000 | train_loss: 1.1324 | train_metric: 0.5584
step: 61000/250000 | train_loss: 1.1301 | train_metric: 0.5587
step: 62000/250000 | train_loss: 1.1323 | train_metric: 0.5589
step: 63000/250000 | train_loss: 1.1303 | train_metric: 0.5653
step: 64000/250000 | train_loss: 1.1306 | train_metric: 0.5642
step: 65000/250000 | train_loss: 1.1305 | train_metric: 0.5629
step: 66000/250000 | train_loss: 1.1303 | train_metric: 0.5634
step: 67000/250000 | train_loss: 1.1185 | train_metric: 0.5638
step: 68000/250000 | train_loss: 1.1178 | train_metric: 0.5640
```

```
pretrained_model_path = LOG_DIR / pretrained_model.__class__.__name__ / 'best' / 'pretrained_lorentz_part_v1.pt'

pretrained_model_path.parent.mkdir(parents=True, exist_ok=True)

torch.save(pretrained_model.state_dict(), pretrained_model_path)

print(f" model saved to: {pretrained_model_path}")
```

```
step: 79000/250000 | train_loss: 1.0999 | train_metric: 0.5715
step: 80000/250000 | train_loss: 1.1000 | train_metric: 0.5706
step: 81000/250000 | train_loss: 1.1012 | train_metric: 0.5706
 model saved to: /content/CMS/MAEs/Hybrid_Transformer_Thanh_Nguyen/logs/LorentzParT/best/pretrained_lorentz_part_v1.pt
```

```
from google.colab import files
files.download(str(pretrained_model_path))
```

```
step: 85000/250000 | train_loss: 1.1001 | train_metric: 0.5714
step: 86000/250000 | train_loss: 1.0999 | train_metric: 0.5715
step: 87000/250000 | train_loss: 1.0998 | train_metric: 0.5714
```

```
# Evaluate the model on the test set
test_loss, test_metric, y_true, y_pred = trainer.evaluate(
    loss_type='cross_entropy',
    plot=[plot_roc_curve, plot_confusion_matrix]
)
```

```
step: 93000/250000 | train_loss: 1.0911 | train_metric: 0.5747
step: 94000/250000 | train_loss: 1.0907 | train_metric: 0.5749
```